PERTEMUAN 6 · PEMROGRAMAN MOBILE FLUTTER

State Management Fundamentals

Stateless vs Stateful · setState() · BLoC

```dart
main.dart

void tekantombol() {
  setState(() {
    number++;
  });
}
```

Nova Agustina, S.T., M.Kom.
Universitas Teknologi Bandung

ALUR MATERI

Empat langkah: dari widget sampai BLoC

01

Stateless - Stateful

Kapan sebuah widget boleh berubah

02

setState()

Memicu rebuild satu widget

03

State management

Memisahkan logic dari tampilan

04

BLoC

Event masuk, State keluar

Target praktik Satu aplikasi counter, dua cara: **`setState()`** dan **BLoC**

Widget menggambar UI dari state saat ini

$$UI = f(state)$$

Semua komponen Flutter adalah **widget**. Widget mendeskripsikan tampilan berdasarkan **state** (kondisi) saat ini.



Stateless tetap, Stateful bisa berubah

Stateless

ketik `stless`

```
const Text('Halo, UTB')
```

SIFAT	State **tidak berubah** setelah widget dibuat
CONTOH	**Text**, **Image**, **Icon**
UPDATE	Harus membuat **instance baru** dari widget
STRUKTUR	**1 class**: extends **`StatelessWidget`**

Stateful

ketik `stful`

☑ **Checkbox** value: false → `true`

SIFAT	State **bisa berubah** saat aplikasi berjalan
CONTOH	**Checkbox**, **TextField**, tombol yang mengubah widget lain
UPDATE	Panggil **`setState()`**, widget digambar ulang
STRUKTUR	**2 class**: **`StatefulWidget`** + **`State`**

Stateful widget selalu terdiri dari dua class

main.dart

```dart
class MyApp extends StatefulWidget {
  const MyApp({super.key});
  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  int number = 1;
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(child: Text('$number')),
      ),
    );
  }
}
```

1 **extends StatefulWidget**
Class 1: konfigurasi widget, tidak berubah

2 **createState()**
Membuat objek State untuk widget ini

3 **class _MyAppState**
Class 2: menyimpan data dan build()

4 **Text('$number')**
Membaca state; nilai baru tampil saat rebuild

setState() memberi tahu Flutter untuk menggambar ulang

main.dart

```dart
class _MyAppState extends State<MyApp> {
  int number = 1;
  void tekantombol() {
    setState(() {
      number++;
    });
  }

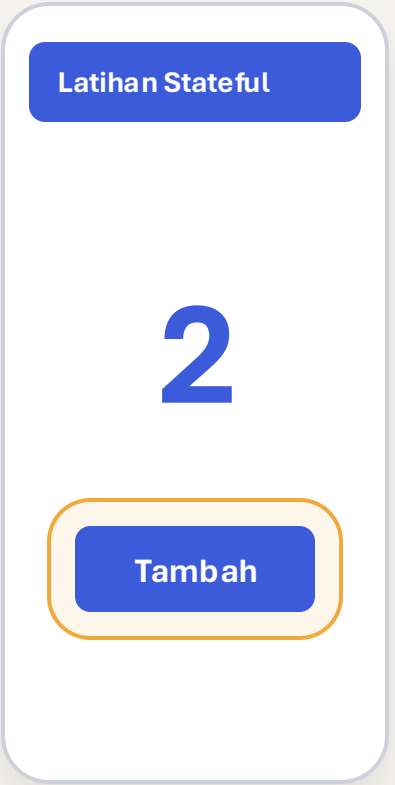
  Widget build(BuildContext context) {
    return Column(children: [
      Text(number.toString()),
      MaterialButton(
        onPressed: tekantombol,
        child: const Text("Tambah"),
      ),
    ]);
  }
}
```

1 **`onPressed: tekantombol`**
Tombol ditekan, fungsi dipanggil

2 **`void tekantombol()`**
Fungsi yang mengubah data

3 **`setState(() {...})`**
Ubah number di dalamnya, lalu Flutter menjadwalkan rebuild

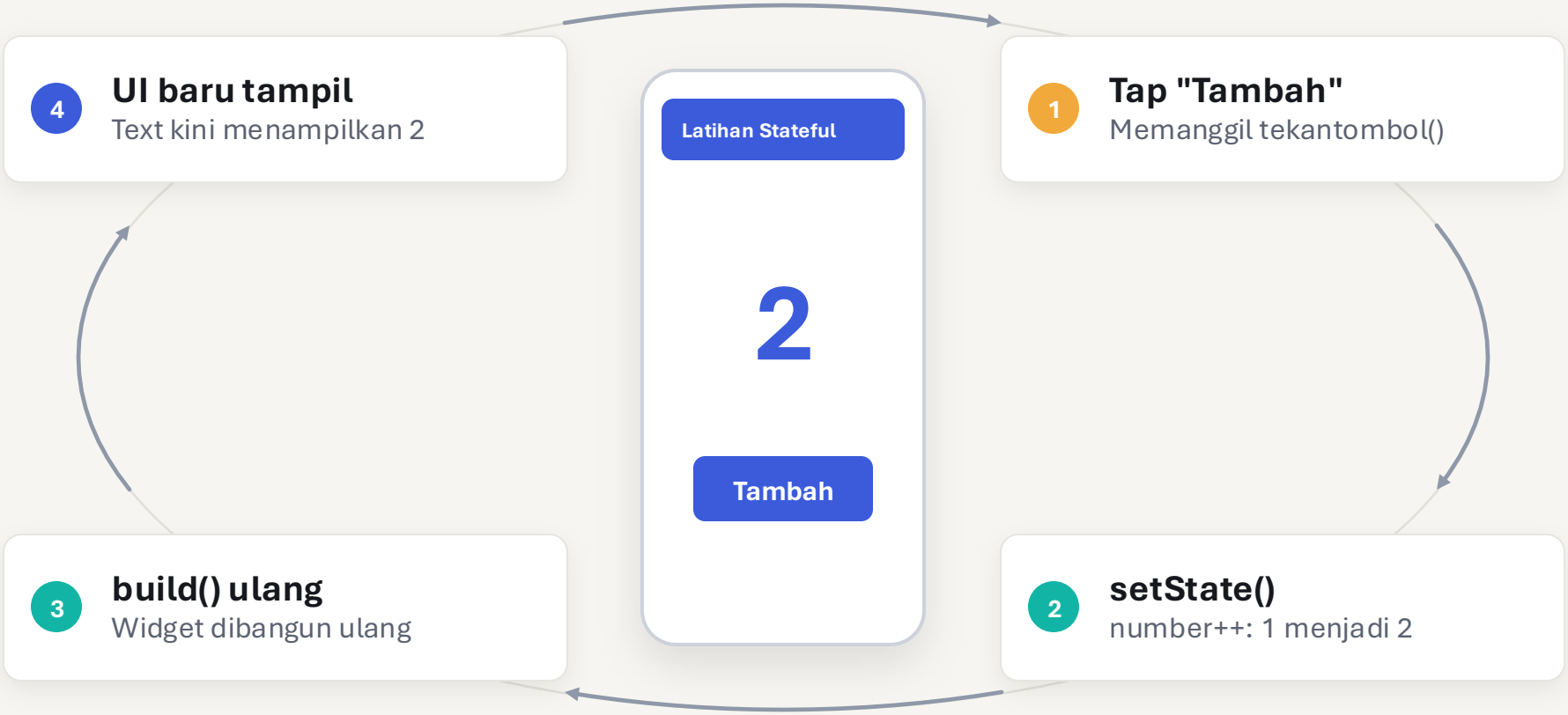
4 **`Text(number.toString())`**
build() jalan lagi, angka baru tampil



tampilan aplikasi

`Tanpa setState(),` number tetap berubah, tetapi layar tidak diperbarui.

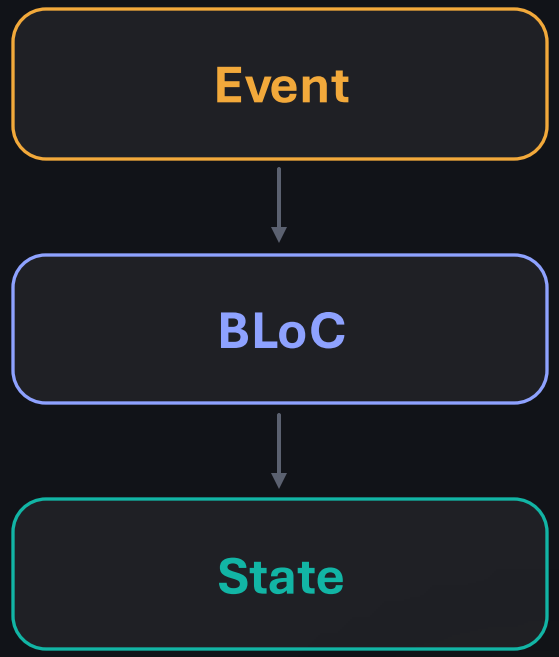
Satu tap memicu satu siklus rebuild



BAGIAN 2 · 03 – 04

Flutter BLoC

Business Logic Component: logika aplikasi dipisah dari tampilan



State management memisahkan logic dari tampilan

VIEW

Tampilan (UI)

Hanya menampilkan state dan mengirim aksi pengguna

LOGIC

Business logic

Mengolah data; bisa dipakai ulang di banyak layar

DATA

State / data

Kondisi aplikasi saat ini

DEKLARATIF

Perubahan state menjadi pemicu UI digambar ulang.

Kode tidak mengubah widget satu per satu; Flutter me-rebuild otomatis dari state terbaru.



Ada banyak pilihan; materi ini memakai BLoC

01

Provider

Paling sederhana; dasar banyak solusi lain

02

Riverpod

Penerus Provider, lebih aman saat compile

03

Redux

Satu store global, alur data searah

04

BLoC

Event → State melalui stream

05

GetX

Ringkas: state, routing, dependency

SOROTAN

Kenapa Provider populer?

Teknik mengolah state yang paling mudah dipelajari.

1

Resource lebih sederhana

Alokasi dan pembuangan objek diatur otomatis

2

Lazy-loading

Data baru dibuat saat pertama dipakai

3

Lebih sedikit boilerplate

Tidak menulis kode berulang untuk tiap class

BLoC mengubah Event menjadi State lewat stream



Intinya UI tidak mengubah data sendiri. UI hanya **mengirim Event** dan **mendengarkan State**.

Tiga langkah menyiapkan BLoC di proyek

1 **Tambahkan dependency**

Jalankan di terminal proyek latihan_bloc

```
$ flutter pub add flutter_bloc equatable
```

2 **Pasang ekstensi VS Code**

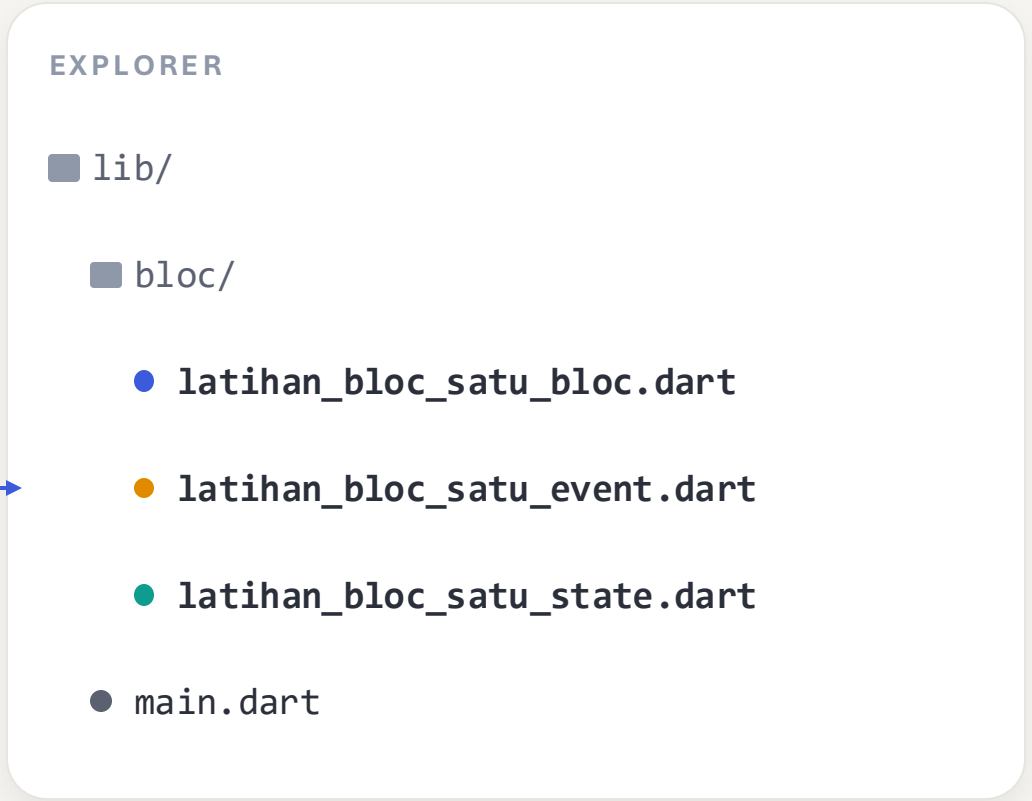
bloc (Felix Angelov) dan Pubspec Assist

bloc | Pubspec Assist

3 **Buat BLoC baru**

Klik kanan lib → Bloc: New Bloc → latihan_bloc_satu

3 file: **bloc** (logika), **event** (perintah), **state** (kondisi)



Event adalah perintah yang dikirim UI ke BLoC

event.dart | state.dart | bloc.dart | main.dart

```dart
part of 'latihan_bloc_satu_bloc.dart';

sealed class LatihanBlocSatuEvent extends Equatable {
  const LatihanBlocSatuEvent();
  @override
  List<Object> get props => [];
}

// Event penambah counter
class IncrementEvent extends LatihanBlocSatuEvent {}

// Event pengurang counter
class DecrementEvent extends LatihanBlocSatuEvent {}
```

1. **part of**
   Bagian dari file bloc; ikut import-nya
2. **sealed class ...Event**
   Induk semua event di BLoC ini
3. **IncrementEvent**
   Perintah: tambah counter
4. **DecrementEvent**
   Perintah: kurangi counter

Setiap event **extends LatihanBlocSatuEvent**, sehingga BLoC menerima semuanya lewat satu tipe.

State adalah kondisi yang dikeluarkan BLoC ke UI

event.dart | **state.dart** | bloc.dart | main.dart

```dart
part of 'latihan_bloc_satu_bloc.dart';

sealed class LatihanBlocSatuState extends Equatable {
  const LatihanBlocSatuState();
  @override
  List<Object> get props => [];
}

final class LatihanBlocSatuInitial
    extends LatihanBlocSatuState {
  final int counter;
  const LatihanBlocSatuInitial(this.counter);
  @override
  List<Object> get props => [counter];
}
```

1. **`sealed class ...State`**
   Induk semua state di BLoC ini
2. **`LatihanBlocSatuInitial`**
   State awal yang dipakai counter
3. **`final int counter`**
   Data yang nanti ditampilkan UI
4. **`props => [counter]`**
   Equatable membandingkan counter; state sama tidak memicu rebuild

BLoC menerima Event, lalu emit State baru

event.dart | state.dart | **bloc.dart** | main.dart

```dart
import 'package:bloc/bloc.dart';
import 'package:equatable/equatable.dart';
part 'latihan_bloc_satu_event.dart';
part 'latihan_bloc_satu_state.dart';

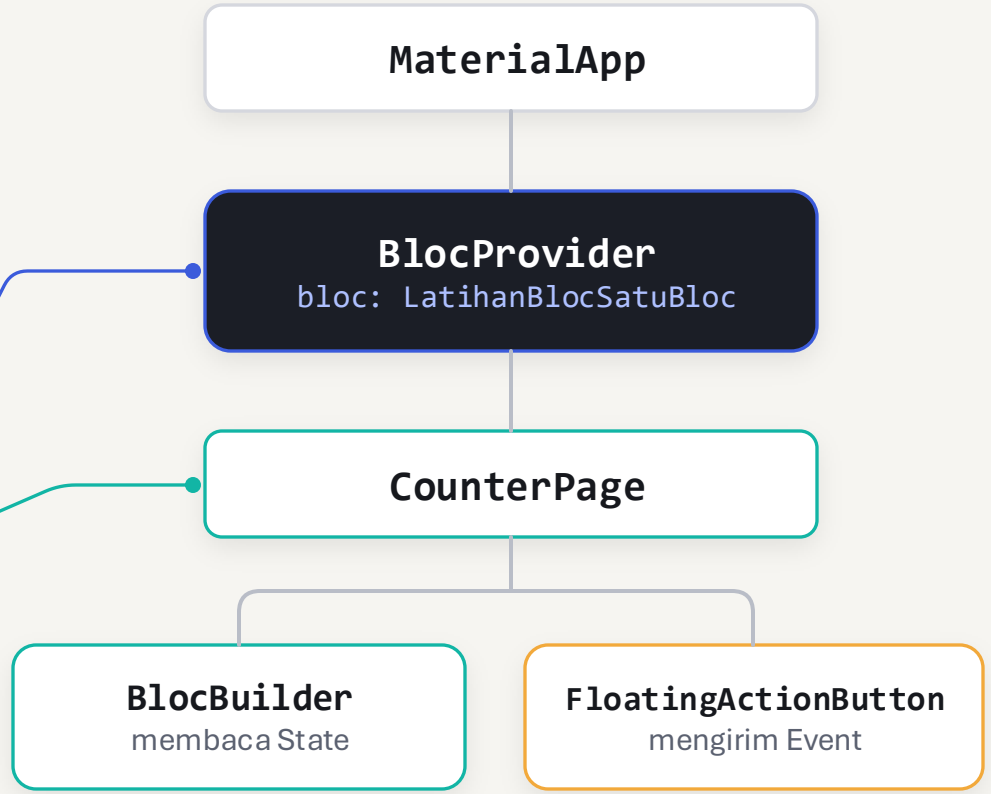
class LatihanBlocSatuBloc
    extends Bloc<LatihanBlocSatuEvent, LatihanBlocSatuState> {
  LatihanBlocSatuBloc() : super(LatihanBlocSatuInitial(0)) {
    on<IncrementEvent>((event, emit) {
      final current = (state as LatihanBlocSatuInitial).counter;
      emit(LatihanBlocSatuInitial(current + 1));
    });
    on<DecrementEvent>((event, emit) {
      final current = (state as LatihanBlocSatuInitial).counter;
      emit(LatihanBlocSatuInitial(current - 1));
    });
  }
}
```

1. **`part '..._event/_state'`**
   Menggabungkan file event dan state
2. **`Bloc<Event, State>`**
   Tipe input (Event) dan output (State)
3. **`super(...Initial(0))`**
   State awal: counter = 0
4. **`on<IncrementEvent>`**
   Handler yang jalan saat event diterima
5. **`emit(...current + 1)`**
   Kirim state baru; UI yang mendengarkan ikut rebuild

BlocProvider menyediakan BLoC untuk widget di bawahnya

event.dart state.dart bloc.dart **main.dart**

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Latihan Bloc",
      home: BlocProvider(
        create: (_) => LatihanBlocSatuBloc(),
        child: const CounterPage(),
      ),
    );
  }
}
```



Anak BlocProvider mengakses BLoC lewat **context**

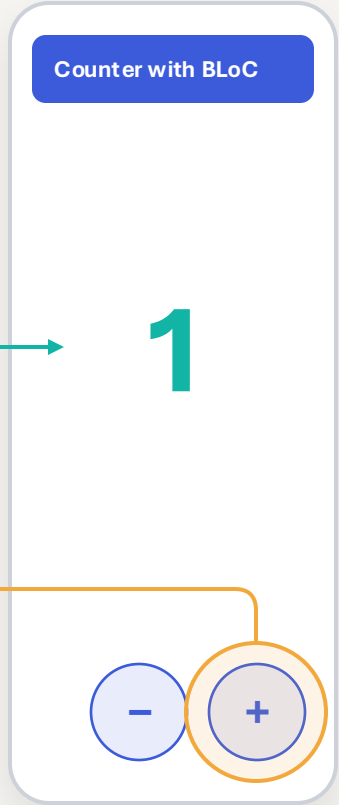
UI mengirim Event dan mendengarkan State

event.dart state.dart bloc.dart **main.dart**

```dart
body: Center(
  child: BlocBuilder<LatihanBlocSatuBloc, LatihanBlocSatuState>(
    builder: (context, state) {
      final counter = (state as LatihanBlocSatuInitial).counter;
      return Text('$counter', style: TextStyle(fontSize: 24));
    },
  ),
),
floatingActionButton: Row(children: [
  FloatingActionButton(
    onPressed: () => context.read<LatihanBlocSatuBloc>()
        .add(DecrementEvent()),
    child: const Icon(Icons.remove)),
  FloatingActionButton(
    onPressed: () => context.read<LatihanBlocSatuBloc>()
        .add(IncrementEvent()),
    child: const Icon(Icons.add)),
]),
```

1 **BlocBuilder**
Rebuild Text saat State baru datang

2 **add(IncrementEvent)**
Tap + mengirim Event ke BLoC



Satu tap + menjalankan lima langkah BLoC



setState()

LOGIC Di dalam widget

COCOK State lokal sederhana: counter, toggle

RISIKO Logic menumpuk saat aplikasi membesar

BLoC

LOGIC Terpisah dari tampilan

COCOK Aplikasi besar, banyak layar, tim

NILAI Logic bisa diuji dan dipakai ulang

PERTEMUAN 6 · STATE MANAGEMENT FUNDAMENTALS

Terima kasih

Nova Agustina, S.T., M.Kom.

Telp 0813 2032 9095

Email nova@utb-univ.ac.id

latihan_bloc_satu_bloc.dart

```dart
// Latihan berikutnya
on<DecrementEvent>((event, emit) {
  final current =
    (state as LatihanBlocSatuInitial).counter;
  emit(LatihanBlocSatuInitial(current - 1));
});
```